Tab 1

```c
#include <stdio.h>
#include <stdlib.h>

/* Node structure */
struct node {
    int data;
    struct node *next;
};

struct node *head = NULL;

/* Function declarations */
void create();
void insert();
void delete();
void traverse();

/* Main function */
int main() {
    int choice;

    while (1) {
        printf("\n--- Circular Linked List Menu ---\n");
        printf("1. Create\n");
        printf("2. Insert\n");
        printf("3. Delete\n");
        printf("4. Traverse\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: create(); break;
            case 2: insert(); break;
            case 3: delete(); break;
            case 4: traverse(); break;
            case 5: exit(0);
            default: printf("Invalid choice\n");
        }
    }
    return 0;
}

/* Create circular linked list */
```

```
void create() {
    int n, i, val;
    struct node *newnode, *temp;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        newnode = (struct node*)malloc(sizeof(struct node));
        printf("Enter data: ");
        scanf("%d", &val);

        newnode->data = val;

        if (head == NULL) {
            head = newnode;
            newnode->next = head;
        } else {
            temp = head;
            while (temp->next != head)
                temp = temp->next;

            temp->next = newnode;
            newnode->next = head;
        }
    }
}

/* Insert at end */
void insert() {
    int val;
    struct node *newnode, *temp;

    newnode = (struct node*)malloc(sizeof(struct node));
    printf("Enter value to insert: ");
    scanf("%d", &val);

    newnode->data = val;

    if (head == NULL) {
        head = newnode;
        newnode->next = head;
    } else {
        temp = head;
```

```
        while (temp->next != head)
            temp = temp->next;

        temp->next = newnode;
        newnode->next = head;
    }
}

/* Delete node by value */
void delete() {
    int val;
    struct node *temp, *prev;

    if (head == NULL) {
        printf("List is empty\n");
        return;
    }

    printf("Enter value to delete: ");
    scanf("%d", &val);

    temp = head;
    prev = NULL;

    /* If head node is to be deleted */
    if (temp->data == val) {
        if (temp->next == head) {
            free(temp);
            head = NULL;
            printf("Node deleted\n");
            return;
        }
        prev = head;
        while (prev->next != head)
            prev = prev->next;

        prev->next = temp->next;
        head = temp->next;
        free(temp);
        printf("Node deleted\n");
        return;
    }

    prev = temp;
```

```
    temp = temp->next;

    while (temp != head && temp->data != val) {
        prev = temp;
        temp = temp->next;
    }

    if (temp == head) {
        printf("Value not found\n");
    } else {
        prev->next = temp->next;
        free(temp);
        printf("Node deleted\n");
    }
}

/* Traverse circular linked list */
void traverse() {
    struct node *temp;

    if (head == NULL) {
        printf("List is empty\n");
        return;
    }

    printf("Circular Linked List: ");
    temp = head;
    do {
        printf("%d -> ", temp->data);
        temp = temp->next;
    } while (temp != head);
    printf("(back to head)\n");
}
```

Compile Result

```
--- Circular Linked List Menu ---
1. Create
2. Insert
3. Delete
4. Traverse
5. Exit
Enter your choice: 1
Enter number of nodes: 1
Enter data: 2

--- Circular Linked List Menu ---
1. Create
2. Insert
3. Delete
4. Traverse
5. Exit
Enter your choice: 2
Enter value to insert: 3

--- Circular Linked List Menu ---
1. Create
2. Insert
```

Compile Result

```
--- Circular Linked List Menu ---
1. Create
2. Insert
3. Delete
4. Traverse
5. Exit
Enter your choice:
```

Compile Result

```
--- Circular Linked List Menu ---
1. Create
2. Insert
3. Delete
4. Traverse
5. Exit
Enter your choice: 2
Enter value to insert: 1

--- Circular Linked List Menu ---
1. Create
2. Insert
3. Delete
4. Traverse
5. Exit
Enter your choice: 3
Enter value to delete: 1
Node deleted

--- Circular Linked List Menu ---
1. Create
2. Insert
```

Compile Result

```
--- Circular Linked List Menu ---
1. Create
2. Insert
3. Delete
4. Traverse
5. Exit
Enter your choice: 2
Enter value to insert: 2

--- Circular Linked List Menu ---
1. Create
2. Insert
3. Delete
4. Traverse
5. Exit
Enter your choice: 4
Circular Linked List: 2 -> (back to head)

--- Circular Linked List Menu ---
1. Create
2. Insert
3. Delete
```

Tab 2